Data Engineering Pipeline Using Serverless SQL

Task Objective

Build a data engineering pipeline to ingest data from an external source, transform it, and prepare it for analysis**.**

Data

- **Link:** https://www.kaggle.com/datasets/andrexibiza/grocery-sales-dataset

Database Schema

The dataset consists of seven interconnected tables:

File Name	Description
`categories.csv`	Defines the categories of the products.
`cities.csv`	Contains city-level geographic data.
`countries.csv`	Stores country-related metadata.
`customers.csv`	Contains information about the customers who make purchases.
`employees.csv`	Stores details of employees handling sales transactions.
`products.csv`	Stores details about the products being sold.
`sales.csv`	Contains transactional data for each sale.

Tools and Technologies

- Azure Data Lake Storage (ADLS)
    - Store raw and transform data using medallion architecture
- Azure Synapse Analytics
    - To ingest transform and prepare for analysis

Azure Data Lake Storage (ADLS)

categories.csv	2/6/2025, 1:35:45 PM	Hot (Inferred)	Block blob	137 B	Available	•••
cities.csv	2/6/2025, 1:35:45 PM	Hot (Inferred)	Block blob	2.12 KiB	Available	•••
countries.csv	2/6/2025, 1:35:45 PM	Hot (Inferred)	Block blob	3.1 KiB	Available	•••
customers.csv	2/6/2025, 1:35:46 PM	Hot (Inferred)	Block blob	4.23 MiB	Available	•••
employees.csv	2/6/2025, 1:35:45 PM	Hot (Inferred)	Block blob	1.67 KiB	Available	•••
products.csv	2/6/2025, 1:35:45 PM	Hot (Inferred)	Block blob	35.98 KiB	Available	•••
sales.csv	2/6/2025, 1:37:13 PM	Hot (Inferred)	Block blob	493.07 MiB	Available	•••

Azure Synapse Analytics

1. **Data Ingestion (bronze):**
   - **Create a database and schema:**

```
--create a database
CREATE DATABASE SALES_DW;

USE SALES_DW;

ALTER DATABASE SALES_DW COLLATE Latin1_General_100_CI_AI_SC_UTF8;

-- CREATE SCHEMA:
CREATE SCHEMA bronze;

CREATE SCHEMA silver;

CREATE SCHEMA gold;
```

- **Create a external data source and file format:**

```
-- EXTERNAL DATA SOURCE
IF NOT EXISTS (SELECT * FROM sys.external_data_sources WHERE name = 'sales_data_raw')
    CREATE EXTERNAL DATA SOURCE sales_data_raw
    WITH (
            LOCATION = 'abfss://project-data@synapsedatasetadls.dfs.core.windows.net/sales-data',

        );

-- create a csv file format
IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name ='csv_file_format_pv1')
  CREATE EXTERNAL FILE FORMAT csv_file_format_pv1
  WITH (
      FORMAT_TYPE = DELIMITEDTEXT,
      FORMAT_OPTIONS (
        FIELD_TERMINATOR = ','
      , STRING_DELIMITER = '"'
      , First_Row = 2
      , USE_TYPE_DEFAULT = FALSE
      , Encoding = 'UTF8'
      , PARSER_VERSION = '1.0' )
      );

IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name ='csv_file_format_pv2')
  CREATE EXTERNAL FILE FORMAT csv_file_format_pv2
  WITH (
      FORMAT_TYPE = DELIMITEDTEXT,
      FORMAT_OPTIONS (
        PARSER_VERSION = '2.0',
        FIRST_ROW = 2,
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        USE_TYPE_DEFAULT = FALSE )
      );
```

- **Add raw data to the external table in bronze schema:**

  Categories table:

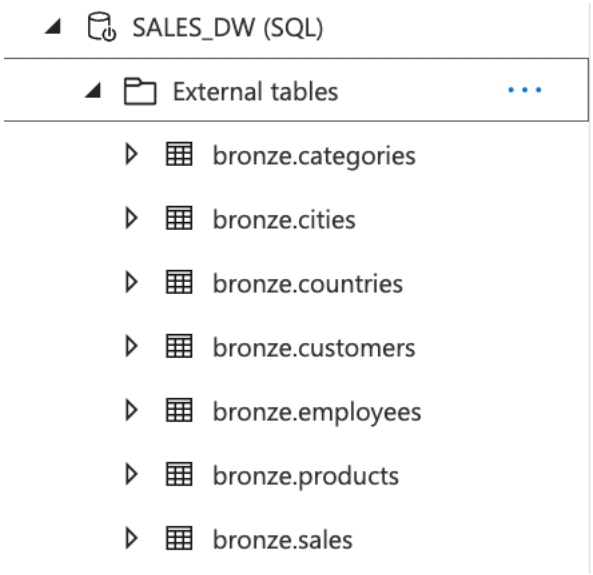
```
-- Categories Table
CREATE EXTERNAL TABLE bronze.categories (
    CategoryID INT,
    CategoryName NVARCHAR(100)
)
WITH (
    LOCATION = 'categories.csv',
    DATA_SOURCE = sales_data_raw,
    FILE_FORMAT = csv_file_format_pv1
);
```

Product table:

```
-- Products Table
CREATE EXTERNAL TABLE bronze.products (
    ProductID bigint,
    ProductName nvarchar(4000),
    Price float,
    CategoryID bigint,
    Class nvarchar(4000),
    ModifyDate date,
    Resistant nvarchar(4000),
    IsAllergic nvarchar(4000),
    VitalityDays float
    )
    WITH (
    LOCATION = 'products.csv',
    DATA_SOURCE = sales_data_raw,
    FILE_FORMAT = csv_file_format_pv2
    )
GO

SELECT * FROM bronze.products;
```

ProductID	ProductName	Price	CategoryID	Class	ModifyDate	Resistant	IsAllergic	VitalityDays
1	Flour - Whole Wheat	74.2988	3	Medium	2018-02-16	Durable	Unknown	0
2	Cookie Chocolate Chip With	91.2329	3	Medium	2017-02-12	Unknown	Unknown	0
3	Onions - Cippolini	9.1379	9	Medium	2018-03-15	Weak	False	111
4	Sauce - Gravy, Au Jus, Mix	54.3055	9	Medium	2017-07-16	Durable	Unknown	0
5	Artichokes - Jerusalem	65.4771	2	Low	2017-08-16	Durable	True	27
6	Wine - Magnotta - Cab Sauv	79.7184	8	High	2017-05-25	Unknown	Unknown	0

Create all 7 external tables in bronze schema



Query sales table to see the data.

SalesID	SalesPersonID	CustomerID	ProductID	Quantity	Discount	TotalPrice	SalesDate	TransactionNu...
1	6	27039	381	7	0.00	0.00	2018-02-05	FQL4S94E4ME1...
2	16	25011	61	7	0.00	0.00	2018-02-02	12UGLX40DJ1A...
3	13	94024	23	24	0.00	0.00	2018-05-03	5DT8RCPL87KI...
4	8	73966	176	19	0.20	0.00	2018-04-07	R3DR9MLD5NR...
5	10	32653	310	9	0.00	0.00	2018-02-12	4BGS0Z5OMAZ...
6	13	28663	413	8	0.00	0.00	2018-02-07	3KTAYIZPGDQ...
7	14	46674	370	12	0.00	0.00	2018-03-02	ICRZIHELQCVB...
8	3	12687	287	4	0.20	0.00	2018-01-17	6X9MOQUH92...

2. Data Cleaning and Transformation (SILVER):

Tables are large in size so it is storage expensive.

Solution : to transform the table into parquet file and join tables where necessary.

e.g. linking products to categories and cities to countries.

- silver.countries_cleaned
- silver.customers_cleaned
- silver.employees_cleaned
- silver.products_cleaned
- silver.sales_cleaned

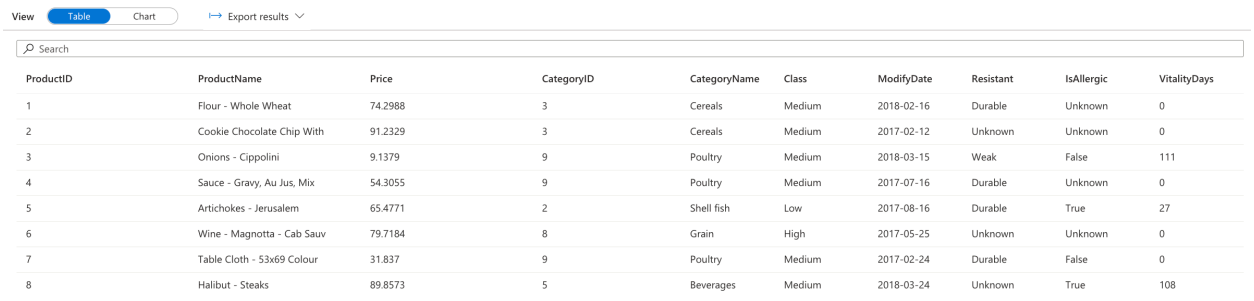
Create a parquet file format:

```
USE SALES_DW;

IF NOT EXISTS (SELECT * FROM sys.external_file_formats WHERE name ='parquet_file_format')
  CREATE EXTERNAL FILE FORMAT parquet_file_format
  WITH (
        FORMAT_TYPE = PARQUET,
        DATA_COMPRESSION = 'org.apache.hadoop.io.compress.SnappyCodec'
       );
```

products_cleaned table:

- Join products and categories table

```
-- Create products Table
IF OBJECT_ID('silver.products_cleaned') IS NOT NULL
    DROP EXTERNAL TABLE  silver.products_cleaned ;

CREATE EXTERNAL TABLE silver.products_cleaned
    WITH (
        DATA_SOURCE = sales_data_raw,
        LOCATION = 'silver/products_cleaned',
        FILE_FORMAT = parquet_file_format
    )
AS
    SELECT
        p.ProductID,
        p.ProductName,
        p.Price,
        c.CategoryID,
        c.CategoryName,
        p.Class,
        TRY_CAST(p.ModifyDate AS DATE) AS ModifyDate,
        p.Resistant,
        p.IsAllergic,
        p.VitalityDays
    FROM
        bronze.products AS p
    LEFT JOIN
        bronze.categories AS c
    ON
        p.CategoryID = c.CategoryID
    WHERE
        p.ProductID IS NOT NULL AND p.Price > 0
```



View Table Chart Export results

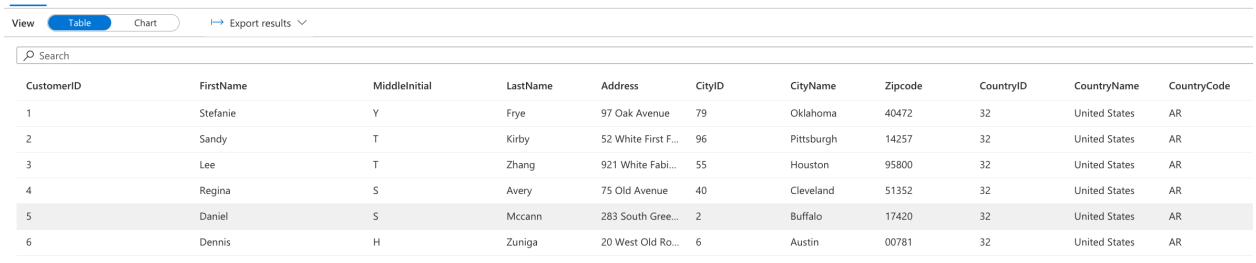
Search

ProductID	ProductName	Price	CategoryID	CategoryName	Class	ModifyDate	Resistant	IsAllergic	VitalityDays
1	Flour - Whole Wheat	74.2988	3	Cereals	Medium	2018-02-16	Durable	Unknown	0
2	Cookie Chocolate Chip With	91.2329	3	Cereals	Medium	2017-02-12	Unknown	Unknown	0
3	Onions - Cippolini	9.1379	9	Poultry	Medium	2018-03-15	Weak	False	111
4	Sauce - Gravy, Au Jus, Mix	54.3055	9	Poultry	Medium	2017-07-16	Durable	Unknown	0
5	Artichokes - Jerusalem	65.4771	2	Shell fish	Low	2017-08-16	Durable	True	27
6	Wine - Magnotta - Cab Sauv	79.7184	8	Grain	High	2017-05-25	Unknown	Unknown	0
7	Table Cloth - 53x69 Colour	31.837	9	Poultry	Medium	2017-02-24	Durable	False	0
8	Halibut - Steaks	89.8573	5	Beverages	Medium	2018-03-24	Unknown	True	108

customers_cleaned table:

- Join customers, countries, cities table

```
--customers table
IF OBJECT_ID('silver.customers_cleaned') IS NOT NULL
    DROP EXTERNAL TABLE silver.customers_cleaned;

CREATE EXTERNAL TABLE silver.customers_cleaned
    WITH (
        DATA_SOURCE = sales_data_raw,
        LOCATION = 'silver/customers_cleaned',
        FILE_FORMAT = parquet_file_format
    )
AS
    SELECT
        c.CustomerID,
        c.FirstName,
        c.MiddleInitial,
        c.LastName,
        c.Address,
        ci.CityID,
        ci.CityName,
        ci.Zipcode,
        co.CountryID,
        co.CountryName,
        co.CountryCode
    FROM
        bronze.customers AS c
    LEFT JOIN
        bronze.cities AS ci
    ON
        c.CityID = ci.CityID
    LEFT JOIN
        bronze.countries AS co
    ON
        ci.CountryID = co.CountryID
    WHERE
        c.CustomerID IS NOT NULL;

select * from silver.customers_cleaned
```



View | Table | Chart | Export results

Search

CustomerID	FirstName	MiddleInitial	LastName	Address	CityID	CityName	Zipcode	CountryID	CountryName	CountryCode
1	Stefanie	Y	Frye	97 Oak Avenue	79	Oklahoma	40472	32	United States	AR
2	Sandy	T	Kirby	52 White First F...	96	Pittsburgh	14257	32	United States	AR
3	Lee	T	Zhang	921 White Fabi...	55	Houston	95800	32	United States	AR
4	Regina	S	Avery	75 Old Avenue	40	Cleveland	51352	32	United States	AR
5	Daniel	S	Mccann	283 South Gree...	2	Buffalo	17420	32	United States	AR
6	Dennis	H	Zuniga	20 West Old Ro...	6	Austin	00781	32	United States	AR

countries_cleaned table:

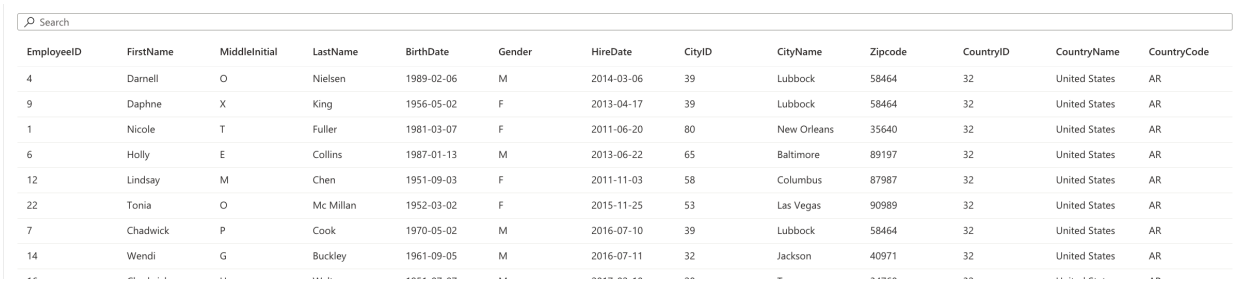
- Join cities and countries table

```sql
-- countries table
IF OBJECT_ID('silver.countries_cleaned') IS NOT NULL
    DROP EXTERNAL TABLE silver.countries_cleaned;

CREATE EXTERNAL TABLE silver.countries_cleaned
    WITH (
        DATA_SOURCE = sales_data_raw,
        LOCATION = 'silver/countries_cleaned',
        FILE_FORMAT = parquet_file_format
    )
AS
    SELECT
        co.CountryID,
        co.CountryName,
        co.CountryCode,
        ci.CityID,
        ci.CityName,
        ci.Zipcode
    FROM
        bronze.countries AS co
    LEFT JOIN
        bronze.cities AS ci
    ON
        co.CountryID = ci.CountryID
    WHERE
        co.CountryID IS NOT NULL;

select * from silver.countries_cleaned;
```

CountryID	CountryName	CountryCode	CityID	CityName	Zipcode
32	United States	AR	48	Long Beach	97859
32	United States	AR	47	San Francisco	00157
32	United States	AR	13	Akron	83448
32	United States	AR	12	Tacoma	43085
32	United States	AR	11	Spokane	38103
32	United States	AR	10	Toledo	52048
32	United States	AR	9	Atlanta	66212

Employees_cleaned

```sql
-- employee table
IF OBJECT_ID('silver.employees_cleaned') IS NOT NULL
    DROP EXTERNAL TABLE silver.employees_cleaned;

CREATE EXTERNAL TABLE silver.employees_cleaned
    WITH (
        DATA_SOURCE = sales_data_raw,
        LOCATION = 'silver/employees_cleaned',
        FILE_FORMAT = parquet_file_format
    )
AS
    SELECT
        e.EmployeeID,
        e.FirstName,
        e.MiddleInitial,
        e.LastName,
        TRY_CAST(e.BirthDate AS DATE) AS BirthDate,
        e.Gender,
        e.HireDate,
        ci.CityID,
        ci.CityName,
        ci.Zipcode,
        co.CountryID,
        co.CountryName,
        co.CountryCode
    FROM
        bronze.employees AS e
    LEFT JOIN
        bronze.cities AS ci
    ON
        e.CityID = ci.CityID
    LEFT JOIN
        bronze.countries AS co
    ON
        ci.CountryID = co.CountryID
    WHERE
        e.EmployeeID IS NOT NULL;
```



Search

EmployeeID	FirstName	MiddleInitial	LastName	BirthDate	Gender	HireDate	CityID	CityName	Zipcode	CountryID	CountryName	CountryCode
4	Darnell	O	Nielsen	1989-02-06	M	2014-03-06	39	Lubbock	58464	32	United States	AR
9	Daphne	X	King	1956-05-02	F	2013-04-17	39	Lubbock	58464	32	United States	AR
1	Nicole	T	Fuller	1981-03-07	F	2011-06-20	80	New Orleans	35640	32	United States	AR
6	Holly	E	Collins	1987-01-13	M	2013-06-22	65	Baltimore	89197	32	United States	AR
12	Lindsay	M	Chen	1951-09-03	F	2011-11-03	58	Columbus	87987	32	United States	AR
22	Tonia	O	Mc Millan	1952-03-02	F	2015-11-25	53	Las Vegas	90989	32	United States	AR
7	Chadwick	P	Cook	1970-05-02	M	2016-07-10	39	Lubbock	58464	32	United States	AR
14	Wendi	G	Buckley	1961-09-05	M	2016-07-11	32	Jackson	40971	32	United States	AR

Sales_cleaned

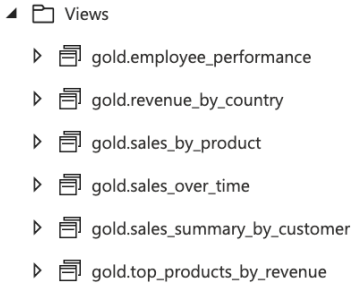
- Join products, sales table
- Derived column revenue

```sql
-- sales table
IF OBJECT_ID('silver.sales_cleaned') IS NOT NULL
    DROP EXTERNAL TABLE silver.sales_cleaned;

CREATE EXTERNAL TABLE silver.sales_cleaned
    WITH (
        DATA_SOURCE = sales_data_raw,
        LOCATION = 'silver/sales_cleaned',
        FILE_FORMAT = parquet_file_format
    )
AS
    SELECT
        s.SalesID,
        s.SalesPersonID,
        s.CustomerID,
        s.ProductID,
        TRY_CAST(s.SalesDate AS DATE) AS SalesDate,
        s.Quantity,
        ROUND(p.Price, 2) AS ProductPrice,
        TRY_CAST((s.Discount*100) AS FLOAT) AS 'Discount(%)',
        ROUND((s.Quantity * p.Price) * (1 - s.Discount), 2) AS Revenue -- derived column,
        s.TransactionNumber
    FROM
        bronze.sales AS s
    LEFT JOIN
        bronze.products AS p
    ON
        s.ProductID = p.ProductID
    WHERE
        s.SalesID IS NOT NULL AND s.Quantity > 0 AND p.ProductID IS NOT NULL;

SELECT * FROM silver.sales_cleaned;
```

SalesID	SalesPersonID	CustomerID	ProductID	SalesDate	Quantity	ProductPrice	Discount(%)	Revenue	TransactionNu...
1	6	27039	381	2018-02-05	7	44.23	0	309.64	FQL4S94E4ME1...
3	13	94024	23	2018-05-03	24	79.02	0	1896.44	5DT8RCPL87KI...
5	10	32653	310	2018-02-12	9	79.98	0	719.8	4BGS0Z5OMAZ...
7	14	46674	370	2018-03-02	12	21.88	0	262.57	ICRZIHELQCVB...
9	16	89009	124	2018-04-27	23	18.29	0	420.65	P0UARL09H66...
11	5	67670	405	2018-03-30	18	0.43	20	6.16	Z2WDS9TJXLA...

3. Data Preparation for Analysis (GOLD):

Crating external views from the tables:



Sales Summary by Customer

```sql
USE SALES_DW;

-- Sales Summary by Customer
IF OBJECT_ID('gold.sales_summary_by_customer') IS NOT NULL
    DROP VIEW gold.sales_summary_by_customer;

CREATE VIEW gold.sales_summary_by_customer AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.MiddleInitial, ' ', c.LastName) AS Customer_FullName,
    c.CityName,
    c.CountryName,
    COUNT(s.SalesID) AS TotalSales,
    SUM(s.Quantity) AS TotalQuantity,
    ROUND(SUM(s.Revenue), 2) AS TotalRevenue
FROM
    silver.sales_cleaned AS s
INNER JOIN
    silver.customers_cleaned AS c
ON
    s.CustomerID = c.CustomerID
GROUP BY
    c.CustomerID, c.FirstName, c.MiddleInitial, c.LastName, c.CityName, c.CountryName;

SELECT * FROM gold.sales_summary_by_customer;
```

Results Messages

View Table Chart Export results

Search

CustomerID	Customer_FullName	CityName	CountryName	TotalSales	TotalQuantity	TotalRevenue
1	Stefanie Y Frye	Oklahoma	United States	65	65	3263.59
2	Sandy T Kirby	Pittsburgh	United States	64	64	3397.94
3	Lee T Zhang	Houston	United States	71	71	3327.84
4	Regina S Avery	Cleveland	United States	69	69	3122.6
5	Daniel S Mccann	Buffalo	United States	59	59	2650.37
6	Dennis H Zuniga	Austin	United States	73	73	4035.67

Sales by Product

```sql
-- Sales by Product
IF OBJECT_ID('gold.sales_by_product') IS NOT NULL
    DROP VIEW gold.sales_by_product;

CREATE VIEW gold.sales_by_product AS
SELECT
    p.ProductID,
    p.ProductName,
    p.CategoryName,
    COUNT(s.SalesID) AS TotalSales,
    SUM(s.Quantity) AS TotalQuantitySold,
    Round(SUM(s.revenue),2) AS TotalRevenue
FROM
    silver.sales_cleaned AS s
INNER JOIN
    silver.products_cleaned AS p
ON
    s.ProductID = p.ProductID
GROUP BY
    p.ProductID, p.ProductName, p.CategoryName;

SELECT * FROM gold.sales_by_product;

-- Emplovee Performance
```

Results Messages

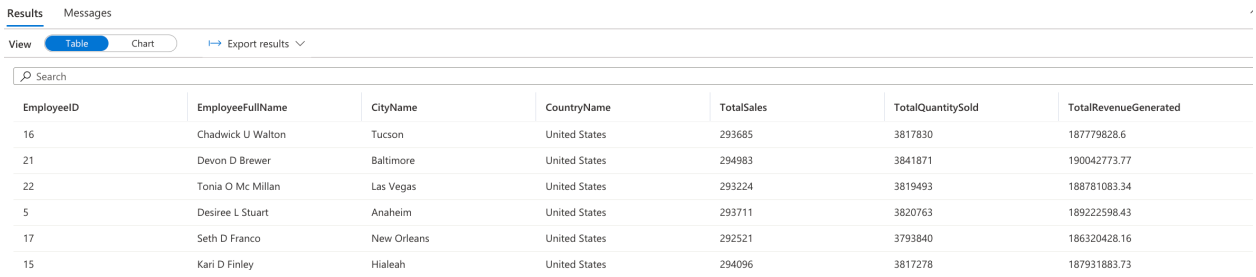
View Table Chart Export results

Search

ProductID	ProductName	CategoryName	TotalSales	TotalQuantitySold	TotalRevenue
1	Flour - Whole Wheat	Cereals	14733	191235	13784693.63
2	Cookie Chocolate Chip With	Cereals	14840	193683	17128629.52
3	Onions - Cippolini	Poultry	14855	193059	1711346.59
4	Sauce - Gravy, Au Jus, Mix	Poultry	15049	195232	10289097.62
5	Artichokes - Jerusalem	Shell fish	14940	194129	12327697.18
6	Wine - Magnotta - Cab Sauv	Grain	14993	195449	15112870.3
7	Table Cloth - 53x69 Colour	Poultry	15073	195278	6028316.1

Employee Performance

```sql
-- Employee Performance
IF OBJECT_ID('gold.employee_performance') IS NOT NULL
    DROP VIEW gold.employee_performance;

CREATE VIEW gold.employee_performance AS
SELECT
    e.EmployeeID,
    CONCAT(e.FirstName, ' ', e.MiddleInitial, ' ', e.LastName) AS EmployeeFullName,
    e.CityName,
    e.CountryName,
    COUNT(s.SalesID) AS TotalSales,
    SUM(s.Quantity) AS TotalQuantitySold,
    ROUND(SUM(s.revenue),2) AS TotalRevenueGenerated
FROM
    silver.sales_cleaned AS s
INNER JOIN
    silver.employees_cleaned AS e
ON
    s.SalesPersonID = e.EmployeeID
GROUP BY
    e.EmployeeID, e.FirstName, e.MiddleInitial, e.LastName, e.CityName, e.CountryName;
```



Results Messages

View Table Chart Export results

Search

EmployeeID	EmployeeFullName	CityName	CountryName	TotalSales	TotalQuantitySold	TotalRevenueGenerated
16	Chadwick U Walton	Tucson	United States	293685	3817830	187779828.6
21	Devon D Brewer	Baltimore	United States	294983	3841871	190042773.77
22	Tonia O Mc Millan	Las Vegas	United States	293224	3819493	188781083.34
5	Desiree L Stuart	Anaheim	United States	293711	3820763	189222598.43
17	Seth D Franco	New Orleans	United States	292521	3793840	186320428.16
15	Kari D Finley	Hialeah	United States	294096	3817278	187931883.73

Revenue by Country

```sql
-- Revenue by Country
IF OBJECT_ID('gold.revenue_by_country') IS NOT NULL
    DROP VIEW gold.revenue_by_country;

CREATE VIEW gold.revenue_by_country AS
SELECT
    co.CountryID,
    co.CountryName,
    co.CityID,
    co.CityName,
    ROUND(SUM(s.revenue),2) AS TotalRevenue,
    COUNT(DISTINCT s.CustomerID) AS UniqueCustomers,
    COUNT(s.SalesID) AS TotalSales
FROM
    silver.sales_cleaned AS s
INNER JOIN
    silver.customers_cleaned AS c
ON
    s.CustomerID = c.CustomerID
INNER JOIN
    silver.countries_cleaned AS co
ON
    c.CityID = co.CityID
GROUP BY
    co.CountryID, co.CountryName, co.CityID, co.CityName;
```

Results Messages

View Table Chart Export results

Search

CountryID	CountryName	CityID	CityName	TotalRevenue	UniqueCustomers	TotalSales
32	United States	18	Little Rock	45207737.77	1053	71769
32	United States	80	New Orleans	42991394.01	964	65736
32	United States	32	Jackson	48416148.23	1065	72532
32	United States	59	Kansas	44836679.62	1029	70218
32	United States	52	Madison	46154612.86	1048	71402
32	United States	68	Anchorage	45659777.9	1039	71000
32	United States	85	St. Petersburg	46316285.62	1055	71906
32	United States	91	Jacksonville	45603141.92	1006	69367

top products by revenue

```sql
-- top products by revenue
IF OBJECT_ID('gold.top_products_by_revenue') IS NOT NULL
    DROP VIEW gold.top_products_by_revenue;

CREATE VIEW gold.top_products_by_revenue AS
SELECT TOP 10
    p.ProductID,
    p.ProductName,
    p.CategoryName,
    ROUND(SUM(s.Revenue),2) AS TotalRevenue,
    SUM(s.Quantity) AS TotalQuantitySold,
    COUNT(s.SalesID) AS TotalSales
FROM
    silver.sales_cleaned AS s
INNER JOIN
    silver.products_cleaned AS p
ON
    s.ProductID = p.ProductID
GROUP BY
    p.ProductID, p.ProductName, p.CategoryName
ORDER BY
    TotalRevenue DESC;
```

Results Messages

View Table Chart Export results

Search

ProductID	ProductName	CategoryName	TotalRevenue	TotalQuantitySold	TotalSales
345	Bread - Calabrese Baguette	Dairy	18868840.18	197257	15077
98	Shrimp - 31/40	Cereals	18721943.4	193234	14887
392	Puree - Passion Fruit	Beverages	18703480.58	194957	14933
104	Tia Maria	Beverages	18685125.46	196210	15105
149	Zucchini - Yellow	Snails	18551658.32	194209	14924

Sales Over Time

```sql
-- Sales Over Time
IF OBJECT_ID('gold.sales_over_time') IS NOT NULL
    DROP VIEW gold.sales_over_time;

CREATE VIEW gold.sales_over_time AS
SELECT
    s.SalesDate,
    ROUND(SUM(s.Revenue),2) AS TotalRevenue,
    SUM(s.Quantity) AS TotalQuantitySold,
    COUNT(s.SalesID) AS TotalSales
FROM
    silver.sales_cleaned AS s
WHERE
    s.SalesDate IS NOT NULL
GROUP BY
    s.SalesDate;

SELECT * FROM gold.sales_over_time ORDER BY SalesDate;
```

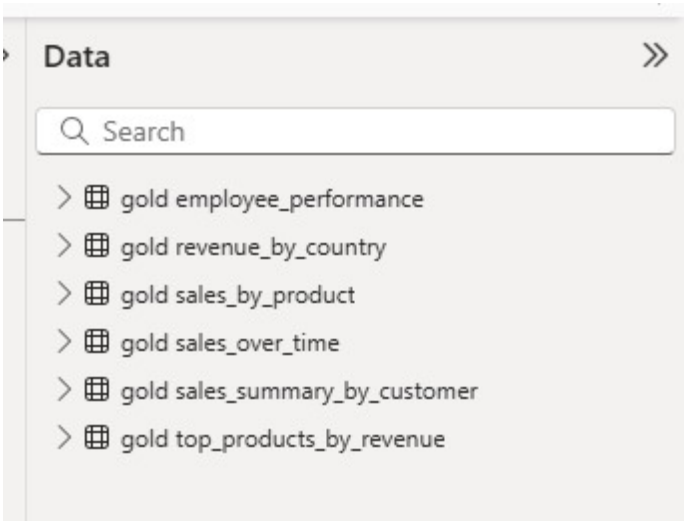
View Table Chart Export results

Search

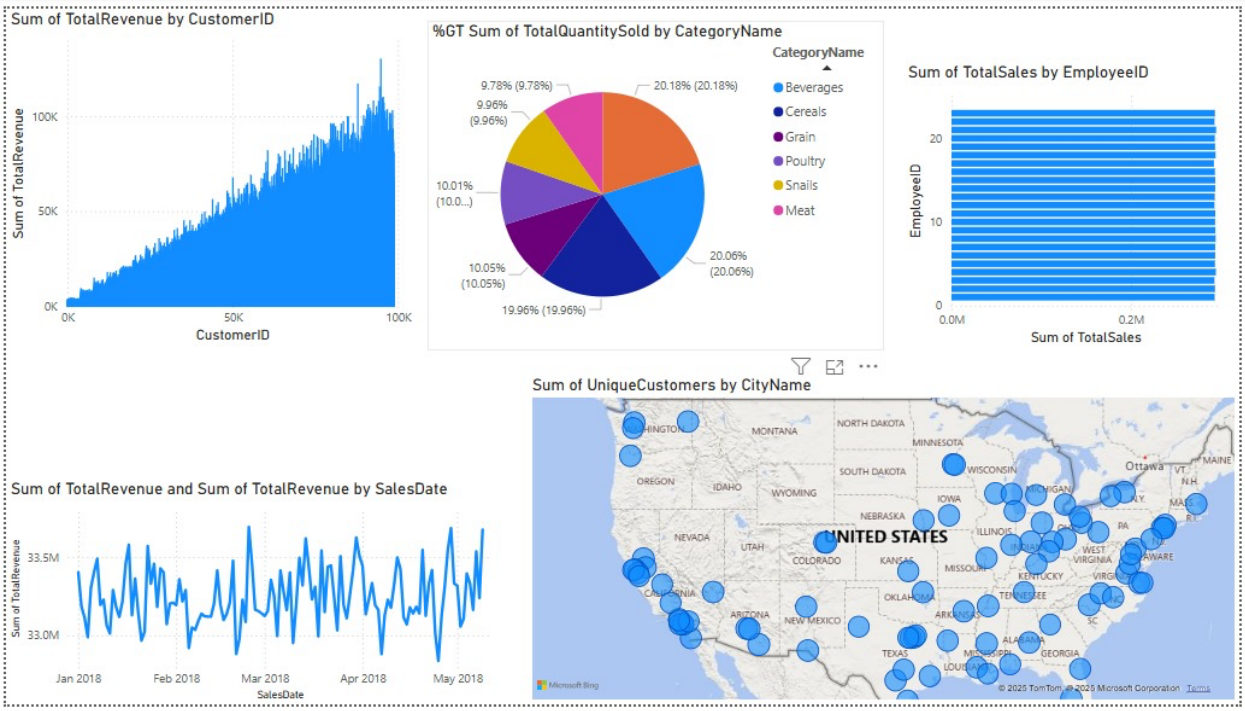
SalesDate	TotalRevenue	TotalQuantitySold	TotalSales
2018-01-01	33403671.63	674961	51856
2018-01-02	33185917.65	675319	52059
2018-01-03	33113884.69	670676	51306
2018-01-04	32989958.85	669381	51527
2018-01-05	33305123.59	678827	52101
2018-01-06	33405345.18	677184	51858
2018-01-07	33491140.03	676302	52089
2018-01-08	33196430.23	672263	51676
2018-01-09	33228740.63	673951	51677

Integrate with power BI DESKTOP

Data:



Sales Dashboard:



Pipeline Design

Pipeline Structure

1. **Pipeline: Bronze Data Ingestion**
    - **Copy Data** activities for each raw CSV file.
    - Target: Bronze layer (data source for external tables).
2. **Pipeline: Silver Transformation**
    - Use **Notebook** for cleaning and joining the bronze tables.
    - Write the cleaned data to ADLS paths (e.g., silver/products_cleaned, silver/customers_cleaned).
3. **Pipeline: Gold View Creation**
    - Use **SQL Script Activities** to run the SQL scripts that create gold views.
4. **Pipeline Scheduling**
    - Schedule pipelines to run in sequence:
        1. Bronze pipeline triggers the silver pipeline after completion.
        2. Silver pipeline triggers the gold view creation pipeline.